Jaroku — Single-User Local App → Multi-Tenant Hosted Platform

Master build spec / Claude Code prompt. Target: ~6,000 concurrent users on a centrally hosted backend. Out of scope for this document: the Tauri desktop wrap (deliberately last, after the client already speaks to a remote authenticated backend).

---

0. STANDING PROCESS REQUIREMENTS — apply to every session in this document

These are hard rules. They apply to every commit you make, in every session below.

1. **NEVER** add any "Generated with Claude Code," attribution footer, or `Co-Authored-By` trailer to any commit message, anywhere, in any form. Every commit must read as authored solely by me. Hard rule, zero exceptions, every commit.
2. Split each session into **exactly 15 separate, genuine, coherent, working commits** — not 4–5 large chunks. Push to `origin/main` after each. If it genuinely can't split that finely without a commit being trivial or fake, tell me honestly and tell me the real number instead of padding it.

Additional rules specific to this migration:

3. **Every commit must leave `npm run typecheck` green on both `server/` and `client/`, and must leave the existing test suites passing.** A commit that breaks a test is not a working commit.
4. **Never modify `schema/events.md`.** Schema v1 is frozen. Everything this migration adds rides *beside* it in new tables, new channels, and new columns on non-schema tables — exactly as pause/resume, the eval engine, and MCP did. The one exception is documented in Session 1 (`workspace_id` as a storage-layer column on `runs` / `steps`), which is a *storage* concern and must never appear in an emitted event.
5. **Do not delete the local single-user path.** Every abstraction introduced here gets two implementations: a `local` one that preserves today's behaviour for development, and a `hosted` one for production. Selected by config, defaulting to `local` . The fixtures, the mock MCP server, and `npm run dev` must keep working with zero cloud dependencies.

6. Before starting a session, read `README.md` in full. Before touching a module, read the module. The README documents *why* each invariant exists; several of them are load-bearing and non-obvious, and a "cleanup" that removes one is a regression.

---

1. Read this first — what is actually changing

Jaroku today is a **local-first, single-tenant workbench**. Three assumptions are baked through the entire codebase, and all three are now false:

Assumption today	Why it dies
One user, one machine, one trust boundary. The person running the server, the person whose API key is in `runtime/.env`, and the person whose generated code gets executed are the same person.	Hosted, they are ~6,000 different people, and one of them is hostile.
The filesystem is the database for anything that isn't a trace. `runtime/agents/<id>/`, `.staging/`, `.history/`, `.checkpoints/`, `runtime/.env` — all local disk, all shared namespace, all assumed to be on the same box as the server.	Multiple stateless server instances, ephemeral run sandboxes, no shared disk.
A run is a subprocess on the host. `uv run python -m jaroku_runner <agent>` spawns *model-written Python* directly on the machine running the control plane.	This is the single largest security problem in the migration. Hosted, it is remote-code-execution-as-a-service against your own infrastructure.

The thing that survives untouched is the trace. `schema/events.md` v1, the NDJSON-on-stdout transport, the stdout guard, `seq` assigned at start and materialised at finish, unknown-cost-is-`null`, cost summed from `steps` not `runs.cost` — all of it is still correct and still the product's primitive. The migration is about *where* code runs and *who* is allowed to see the resulting rows, not about the shape of those rows.

---

2. Decisions to confirm before commit 1

Do not start until these are answered. Ask me and wait; do not pick defaults silently.

D1 — Who pays for model calls?

- **(a) BYOK (recommended default).** Every workspace supplies its own Anthropic/OpenAI key. Encrypted at rest, decrypted only into a run sandbox's environment. Platform spend ≈ 0. Cost accounting stays a reporting feature rather than a billing system.
- **(b) Platform key + credits.** You pay providers, users buy credits. Requires the full metering/reservation/billing stack in Session 6 and makes abuse detection mandatory from day one.
- **(c) Both** — BYOK for power users, a small metered free tier on the platform key for onboarding.

Assume **(c)** with (a) as the enforced default and the platform-key path gated behind a hard per-workspace ceiling, unless I say otherwise. Build the abstraction so the answer is a config flag, not a rewrite.

D2 — Tenancy unit: user or workspace? Assume **workspace**. Every user gets an auto-created personal workspace at signup; a workspace has members with roles. Retrofitting organisations onto a `user_id`-scoped schema later means re-migrating every table and every query, so pay the small cost now. All scoping columns are `workspace_id`, never `user_id`. `user_id` appears only in `workspace_members`, audit rows, and "who did this" attribution columns.

D3 — Auth provider. Clerk or Supabase Auth. Pick Supabase Auth **only** if Postgres is also Supabase; otherwise Clerk, since it is database-agnostic and you will likely want Neon/RDS for the trace tables. Either way the server verifies JWTs against the provider's JWKS and maps the external `sub` to an internal `users.id`. **Never trust a claim the client sent about which workspace it is in.**

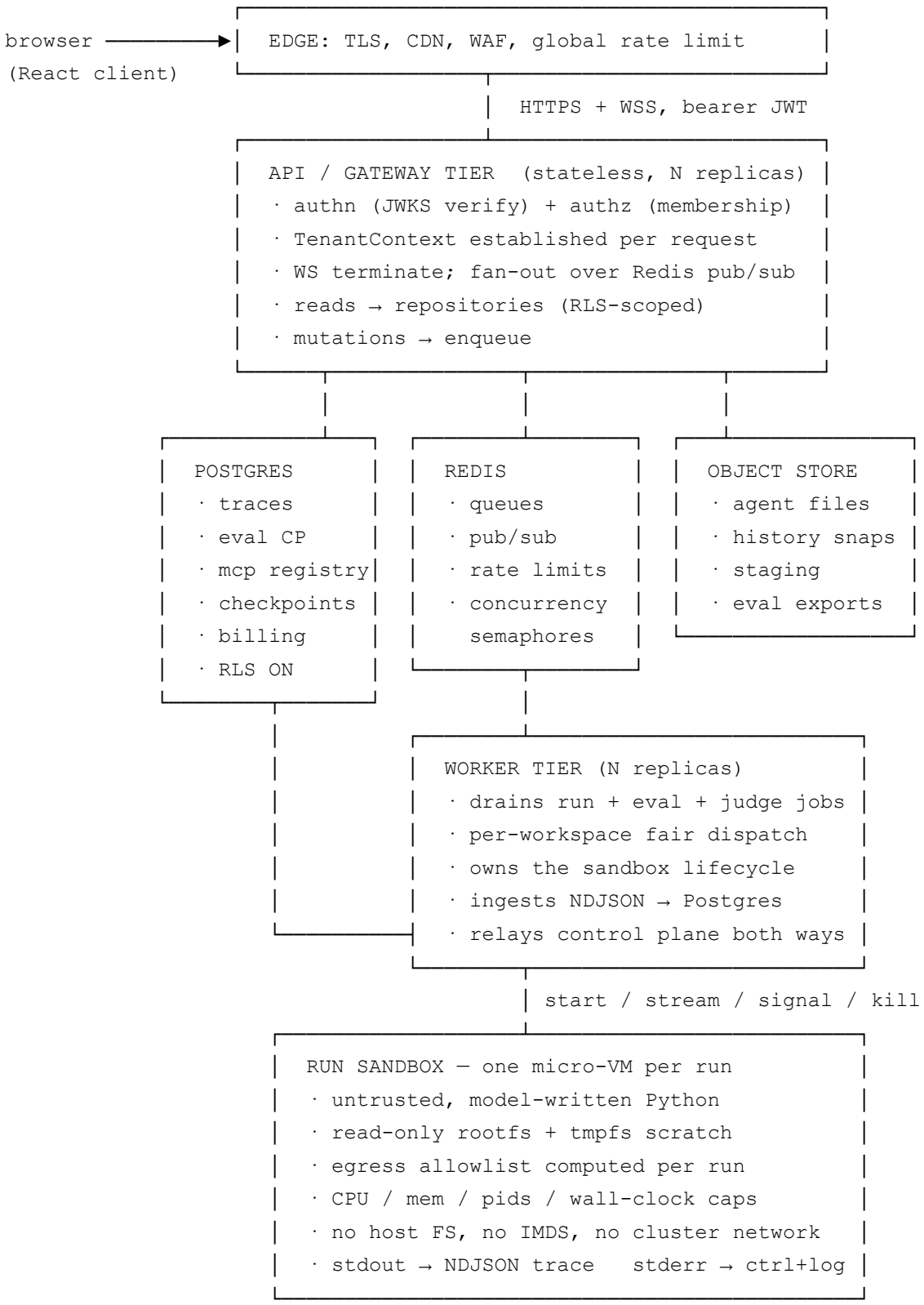
D4 — Sandbox substrate. Fly Machines, Firecracker on AWS (via Fargate is *not* sufficient isolation for untrusted code), gVisor on GKE, or a managed sandbox (E2B, Modal). Recommend **Fly Machines** for the first hosted version: per-run micro-VM, fast cold start, per-machine egress rules, and it matches the deploy targets already in the roadmap. Build behind a `RunSandbox` interface so this is swappable.

D5 — Object storage. S3, R2, or GCS. Recommend **R2** (no egress fees, S3-compatible SDK). Behind an `ObjectStore` interface.

D6 — Scale target reality check. "6,000 concurrent" — concurrent *sessions* (people with the app open, mostly idle) or concurrent *agent runs*? These differ by ~50×. 6,000 open WebSockets is a modest problem. 6,000 simultaneous sandboxed Python VMs is roughly 6,000 × 512 MB = 3 TB of RAM and a serious capacity-planning exercise. Every queue depth, pool size and cap in Session 5 depends on this number. Tell me which one it is.

---

3. Target architecture



4. Invariants that must survive the migration

Treat these as tests, not prose. Session 8 requires a suite that asserts each one.

From the existing README (do not regress):

- `stdout` carries trace events and nothing else. The `dup2(2,1)` guard runs before any generated code is imported.
- The trace never lies: a router classification that cannot be proven is emitted as `state_update`.
- Unknown ≠ zero. Unpriced model → `null` cost. Missing judge criterion → *unscored*, not 0.
- Nothing lands unreviewed: plan before generation, diff before edit, stage + validate + atomic swap.
- Cost is summed from `steps`, never `runs.cost`.
- Generated projects import nothing named `jaroku` and are portable.
- Connector templates and `mcp_bridge.py` are copied byte-for-byte, read-only to the edit loop.
- `readOnlyHint: true` from an MCP server is ignored; impact classification is a ratchet; unknown → high.
- A high-impact MCP call stops for confirmation; timing out **denies**.
- A failed MCP refresh never destroys a working tool list.

New, introduced by this migration:

- **No query reaches Postgres without a workspace scope.** Enforced twice: a repository layer that cannot be called without a `TenantContext`, and Postgres RLS as the backstop.
- **No user-supplied string ever becomes a filesystem path on a shared host.** Object keys are derived server-side from validated ids.
- **No generated code executes on the control plane.** Not for validation, not for graph introspection, not for the import check. All three move into the sandbox.
- **No sandbox can reach anything not on its computed allowlist** — including the cloud metadata endpoint, the database, Redis, other sandboxes, and RFC1918 space.
- **No secret is ever written to a file that outlives a run**, returned to a browser, or logged.

Clients learn `configured: true` and nothing more.

- **Every cross-tenant access attempt is a logged, alertable security event**, not a silent 404.

---

5. Session plan

Eight sessions, 15 commits each. Sessions are strictly ordered; each has an entry gate.

#	Session	Entry gate
1	Tenancy foundation: Postgres + repositories + RLS	D1–D3 answered
2	Auth, membership, and the authenticated client	Session 1 merged, isolation suite green
3	Storage isolation: object store, secret store, Postgres checkpointer	Session 2 merged
4	Sandboxed execution + distributed control plane + trace ingestion	Session 3 merged, D4 answered
5	Queueing, fairness, per-workspace limits, eval fan-out	Session 4 merged, D6 answered
6	Cost metering, budgets, billing	Session 5 merged
7	Connector OAuth + credential vault + MCP at multi-tenant scale	Session 6 merged
8	Security hardening, abuse, data lifecycle, observability, deploy	Session 7 merged

---

SESSION 1 — Tenancy foundation

Goal: every row in the system belongs to a workspace, and it is structurally difficult to write a query that crosses one. No auth yet — a dev middleware injects a fixed workspace so the app keeps working end to end throughout.

Why first: every later session writes rows. Retrofitting tenancy after Sessions 3–7 have added tables means auditing every one of them again.

Design

Migrations. Introduce a real migration tool (`node-pg-migrate` or plain numbered SQL files run by a small `migrate.ts` — prefer the latter, it has no dependency and this codebase already favours zero-dependency tooling). Migrations are forward-only, each numbered and idempotent, applied at boot by exactly one instance holding a Postgres advisory lock.

Two databases, one codebase. `store.ts` and `evalStore.ts` and `mcpStore.ts` currently talk to `node:sqlite` directly. Introduce a thin `Db` interface with `SqliteDb` and `PostgresDb` implementations, selected by `JAROKU_DB_DRIVER=sqlite|postgres` . SQLite stays the default for local dev; the fixtures and the whole free-development path must keep working with no Postgres running.

Repository layer. Every table gets a repository whose every method takes a `TenantContext` as its first argument:

```
type TenantContext = {
  workspaceId: string;      // uuid
  actorUserId: string | null; // null for system jobs
  role: 'owner' | 'admin' | 'member' | 'system';
  requestId: string;        // for audit + log correlation
};
```

Rule to enforce with an eslint rule *and* a test: **no file outside `server/src/db/` may import the raw client.** The repositories are the only path to the database.

RLS as backstop. The app connects as a role that is *not* `BYPASSRLS` and is not the table owner. Every tenant table gets:

```
ALTER TABLE runs ENABLE ROW LEVEL SECURITY;
ALTER TABLE runs FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON runs
  USING (workspace_id = current_setting('app.workspace_id', true)::uuid)
  WITH CHECK (workspace_id = current_setting('app.workspace_id', true)::uuid);
```

The repository layer opens a transaction and issues `SET LOCAL app.workspace_id = $1` before any statement. `SET LOCAL` is transaction-scoped, so this is safe under PgBouncer transaction pooling. A missing setting makes `current_setting(..., true)` return NULL and the policy matches nothing — **fail closed, and that is the point.**

Schema

New tables:

```
CREATE TABLE users (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  external_id   text UNIQUE NOT NULL,     -- auth provider `sub`
  email         citext UNIQUE NOT NULL,
  display_name  text,
  created_at    timestamptz NOT NULL DEFAULT now(),
  deleted_at    timestamptz
);

CREATE TABLE workspaces (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  slug          citext UNIQUE NOT NULL,
  name          text NOT NULL,
  kind          text NOT NULL CHECK (kind IN ('personal','team')),
  plan          text NOT NULL DEFAULT 'free',
  created_at    timestamptz NOT NULL DEFAULT now(),
  deleted_at    timestamptz
);

CREATE TABLE workspace_members (
  workspace_id  uuid NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
  user_id       uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  role          text NOT NULL CHECK (role IN ('owner','admin','member')),
  created_at    timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (workspace_id, user_id)
);

CREATE TABLE audit_log (
  id            bigserial PRIMARY KEY,
  workspace_id  uuid,
  actor_user_id uuid,
  action        text NOT NULL,
  target_type   text,
  target_id     text,
  metadata      jsonb NOT NULL DEFAULT '{}',
  ip            inet,
  created_at    timestamptz NOT NULL DEFAULT now()
);
```

Existing tables gain `workspace_id uuid NOT NULL REFERENCES workspaces(id)`: `runs`, `steps`, `datasets`, `dataset_examples`, `rubrics`, `eval_runs`, `eval_jobs`, `eval_scores`, `mcp_servers`, `mcp_tools`, plus the new `agents` table below.

`agents` becomes a table. Today the agent list is derived by listing `runtime/agents/`. That cannot survive object storage or multiple server replicas:

```
CREATE TABLE agents (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id   uuid NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
  slug           text NOT NULL,           -- the old agent id: ^[a-z][a-z0-9_]{0,
  display_name   text,
  current_version integer NOT NULL DEFAULT 1,
  connectors     jsonb NOT NULL DEFAULT '[]',
  required_env   jsonb NOT NULL DEFAULT '[]',
  creation_cost  numeric,                  -- from jaroku.json; null = unknown, n
  created_by     uuid REFERENCES users(id),
  created_at     timestamptz NOT NULL DEFAULT now(),
  deleted_at     timestamptz,
  UNIQUE (workspace_id, slug)
);

CREATE TABLE agent_versions (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_id      uuid NOT NULL REFERENCES agents(id) ON DELETE CASCADE,
  version       integer NOT NULL,
  manifest      jsonb NOT NULL,   -- {path: {sha256, bytes}} for every file
  created_by    uuid REFERENCES users(id),
  created_at    timestamptz NOT NULL DEFAULT now(),
  UNIQUE (agent_id, version)
);
```

The uniqueness change that will bite you: agent slugs are globally unique today. They become unique *per workspace*. Every place that treats a slug as a global key — `runtime/agents/<id>/`, `graphIntrospect`, `connectors.ts`, the run pool's attribution, `JAROKU_*` env plumbing — must now carry `(workspace_id, slug)` or, better, the `agents.id` uuid. **Prefer threading the uuid; slugs are a display concern.**

Indexes. Every tenant table gets `workspace_id` as the *leading* column of its hot indexes, not a trailing one:

```
CREATE INDEX runs_ws_started ON runs (workspace_id, started_at DESC);
CREATE INDEX steps_ws_run_seq ON steps (workspace_id, run_id, seq);
CREATE INDEX eval_jobs_ws_status ON eval_jobs (workspace_id, eval_run_id, status)
```

`steps` is the table that gets huge. Plan for partitioning by month in Session 8; get the index order right now so it is cheap later.

Type mapping from SQLite. `store.ts` stores JSON payloads as TEXT and parses on the way out "so live and replayed steps are identical shapes." In Postgres use `jsonb` and keep that guarantee by asserting shape equality in a test that runs the same step through both

drivers.

15 commits

1. `migrate.ts` runner: numbered forward-only SQL files, advisory-lock-guarded, `npm run migrate` + boot-time apply. No schema yet, just the mechanism + a smoke migration.
2. `Db` interface with `SqliteDb`; `store.ts` refactored onto it with zero behaviour change. Existing tests still green.
3. `PostgresDb` implementation + connection pool + `docker-compose.yml` for a local Postgres. `JAROKU_DB_DRIVER` switch.
4. Migration 001: `users`, `workspaces`, `workspace_members`, `audit_log`. Repository + tests for each.
5. Migration 002: `workspace_id` added to `runs` and `steps`; `TenantContext` type; `TraceRepository` with mandatory scoping.
6. Migration 003: `workspace_id` on the eval control plane; `EvalRepository`.
7. Migration 004: `workspace_id` on `mcp_servers` / `mcp_tools`; `McpRepository`.
8. Migration 005: `agents` + `agent_versions`. `agents.ts` reads the table instead of the directory listing; the directory becomes a cache the table describes.
9. RLS: enable + force + policies on every tenant table; a dedicated non-owner app role; `SET LOCAL` wired into the repository transaction wrapper.
10. Backfill/import command: an existing local SQLite `jaroku.db` + `runtime/agents/` → a named workspace in Postgres. Idempotent, resumable, dry-run flag.
11. Dev tenancy middleware: `JAROKU_DEV_WORKSPACE` injects a fixed `TenantContext` so every existing WS command keeps working. Every command handler now takes and forwards the context.
12. `wsRelay.ts`: commands carry a context. **Reads answered locally by the relay must now be authorised** — this is the single most dangerous line in the current design when it goes multi-tenant, and this commit is where it stops being a broadcast.
13. Eslint rule + test: nothing outside `server/src/db/` may import a raw DB client; no repository method without a `TenantContext` first parameter.
14. **`npm run test:tenancy`** — the isolation suite. For every WS command, with two workspaces A and B populated: assert that A's context cannot read, mutate, or enumerate any of B's rows, and that a forged `workspace_id` in a payload is ignored in

favour of the context. This suite grows in every subsequent session and is the gate for all of them.

15. Docs: README section on the tenancy model, the repository rule, and the RLS backstop; `CONTRIBUTING` -style note that a new table without `workspace_id` + a policy + a tenancy test will be rejected.

Acceptance: `JAROKU_DB_DRIVER=postgres npm run dev` runs the full existing single-user experience against Postgres inside one dev workspace. `npm run test:tenancy` green. SQLite path unchanged.

SESSION 2 — Auth, membership, and the authenticated client

Goal: real users, real workspaces, real sessions, and a client that cannot see anything it isn't a member of.

Design

HTTP. The server currently serves only `debug-client.html` and a WebSocket. It needs a real HTTP surface: `/healthz`, `/readyz`, `/v1/auth/*`, `/v1/ws-ticket`, and later the OAuth callbacks. Keep it dependency-light (`node:http` + a tiny router, or Fastify if you'd rather — decide in commit 1 and be consistent).

Token verification. Verify the provider JWT against cached JWKS (cache with a TTL and a forced refresh on unknown `kid`). Check `iss`, `aud`, `exp`, `nbf`. Map `sub` → `users.external_id`, creating the user and their personal workspace on first sight, inside one transaction.

WebSocket auth. Browsers cannot set an `Authorization` header on a WebSocket. Do **not** put a long-lived JWT in the query string (it lands in access logs). Instead:

1. Client `POST /v1/ws-ticket` with its bearer token → server returns a single-use, 30-second, workspace-scoped ticket stored in Redis.
2. Client connects `wss://…/ws?ticket=…`; the server atomically consumes it (`GETDEL`), resolves the context, and only then subscribes.
3. Check the `Origin` header on upgrade — WebSockets are not covered by CORS, and this is the actual CSWSH defence.
4. Re-validate on a timer: a socket open for 8 hours must not outlive a revoked

membership. Push a `session` channel event and force reconnect.

Workspace switching is a new socket, not a mutation on the current one. Simpler and impossible to get subtly wrong.

Authorization, not just authentication. Roles: `owner` (billing, delete workspace, manage members), `admin` (manage agents, connectors, MCP servers, budgets), `member` (create/run/edit agents, run evals). Write the capability matrix as data in one module and check against it in one place — scattered role checks are how you get a hole.

Client changes. `lib/socket` gains: token acquisition, ticket exchange, reconnect-with-refresh, and a hard distinction between "disconnected" (retry) and "unauthorised" (stop, show sign-in). `uiStore` gains session + workspace state. Every store must **fully reset on workspace switch** — a stale `traceStore` bleeding run rows across a switch is a cross-tenant leak in the UI even when the server behaved.

15 commits

1. HTTP layer: router, `/healthz`, `/readyz`, structured request logging with `requestId`, error envelope.
2. JWKS client with caching, forced refresh, and negative caching.
3. Token verification middleware producing an `AuthContext`; unit tests with a fixture issuer (expired, wrong `aud`, wrong `iss`, unknown `kid`, `alg: none`).
4. First-sight provisioning: user + personal workspace + owner membership, transactional and idempotent under concurrent first requests.
5. `TenantContext` resolved from `AuthContext` + requested workspace + membership lookup. Membership is cached in Redis with a short TTL and an explicit invalidation on role change.
6. Capability matrix module + `requireCapability()` + tests.
7. `/v1/ws-ticket`; Redis single-use ticket store; WS upgrade path consumes it; origin allowlist check.
8. Every WS command handler wired to a real context; the dev middleware from Session 1 becomes an explicit, loudly-logged `JAROKU_DEV_AUTH=1` escape hatch that refuses to start when `NODE_ENV=production`.
9. Per-socket subscription authorisation: a client is subscribed only to its workspace's channels, and the relay's local read answers are filtered by context.
10. Periodic session revalidation + `session` channel (`expiring`, `revoked`,

`workspace_changed`).

11. Membership management commands: invite, accept, change role, remove. Every one writes an `audit_log` row.
12. Client: auth provider SDK, sign-in/sign-up screens, session store, ticket exchange in `lib/socket`.
13. Client: workspace switcher; **full store reset on switch**; a test asserting no store retains rows across a switch.
14. `npm run test:tenancy` extended: every new command, plus token-level attacks (expired, cross-workspace ticket, revoked-membership socket, forged workspace in payload, ticket replay).
15. Docs + a threat-model note on why the ticket exists and why origin checking is not optional.

Acceptance: two real accounts, two workspaces, both using the app simultaneously against one server, neither able to observe the other by any command, socket, or timing.

SESSION 3 — Storage isolation

Goal: remove every assumption that server, agent files, and checkpoints share a disk.

Design

`ObjectStore` interface — `put`, `get`, `head`, `list(prefix)`, `delete`, `copy`, `presignGet`, `presignPut`. Implementations: `FsObjectStore` (dev, rooted under `runtime/.objects/`) and `S3ObjectStore` (R2/S3). Key layout:

```
ws/<workspace_id>/agents/<agent_id>/v<version>/<path>
ws/<workspace_id>/agents/<agent_id>/staging/<staging_id>/<path>
ws/<workspace_id>/exports/<eval_run_id>.csv
```

Keys are built **only** from validated uuids and paths that already passed `projectFs`'s confinement checks. No user string ever reaches a key un-normalised — object stores do not have `..`, but S3 will happily create a key containing one and your local dev `FsObjectStore` will then write outside its root.

Versioned, immutable agent files. `projectFs.atomicSwap` becomes: write a new `agent_versions` row with a manifest, then bump `agents.current_version` in one

transaction. That single UPDATE is the atomic swap. Undo becomes decrementing to a previous version — cheaper and more robust than the current snapshot copy, and it survives across replicas. Keep the existing linear-history semantics the UI already expects.

`SecretStore` interface — `set(ctx, name, value)`, `getForRun(runId, names[])`, `listNames(ctx)`, `delete(ctx, name)`. There is deliberately **no** `get(ctx, name)` that returns a plaintext value to a request handler; secrets flow only into a run's environment. Implementations: `DotEnvSecretStore` (dev, wrapping today's `envWriter.ts` unchanged) and `KmsSecretStore` (per-workspace data key, envelope-encrypted with a KMS master key, ciphertext in Postgres). Preserve today's rule that a value which cannot round-trip is refused rather than mangled.

`envWriter.ts` keeps its job locally and loses it in production. Do not delete it — it is the local implementation, and its round-trip refusal test is worth keeping.

Checkpointer → Postgres. `debug.py` uses `SqliteSaver` writing `runtime/.checkpoints/<run_id>.sqlite`, and branching *physically copies the file*. Replace with `langgraph-checkpoint-postgres`'s `PostgresSaver`, `thread_id = <run_id>`, and a checkpoint namespace prefixed with the workspace id. Branching becomes a scoped `INSERT … SELECT` of the parent's checkpoint rows up to the target `checkpoint_id` into a new thread — genuinely cleaner than the file copy, and it keeps the guarantee that the parent is never mutated.

Watch two things: the Postgres saver must use its **own connection and its own migration set**, not your repository pool with RLS (LangGraph will not issue `SET LOCAL`); isolate it in its own schema, e.g. `langgraph`, with the workspace id embedded in `thread_id` and access mediated entirely by your code. And `evalCleanup.ts`'s sweep becomes a DELETE by thread prefix — keep the rule that an *interactive* run's checkpoint is never swept.

15 commits

1. `ObjectStore` interface + `FsObjectStore` + key-builder with validation and tests (including the `..`-in-key case).
2. `S3ObjectStore` with retries, multipart for large payloads, and a conformance test suite both implementations must pass.
3. Migration: `agent_versions` manifests populated; `projectFs` reads through `ObjectStore`.
4. Generation path: staging writes to the object store under a staging id; validation reads from there; commit = new version row + `current_version` bump.
5. Edit path: proposal staging, diff generation from object-store reads, apply = version

bump, undo = version pointer move. Linear history preserved.

6. Read-only enforcement re-verified against the new layout: connector templates, `jaroku.json`, top-level `__init__.py`, `mcp_tools.json`, `tools/mcp_bridge.py`. **This must be re-tested, not assumed** — the block list currently matches on local paths.
7. `SecretStore` interface + `DotEnvSecretStore` wrapping `envWriter.ts`, existing tests unchanged.
8. `KmsSecretStore`: per-workspace DEK, envelope encryption, ciphertext + key id + version in Postgres, rotation-capable. Round-trip refusal preserved.
9. `secret_refs` table: names, provider, scope, `configured` flag, `last_used_at`. Never a value. Client sees names and `configured: true` only.
10. Python: `PostgresSaver` in `debug.py` behind `JAROKU_CHECKPOINTER=sqlite|postgres`; thread naming; `langgraph` schema migrations.
11. Branch as a scoped row copy; parent immutability test; resume across a *different* worker process test.
12. `evalCleanup.ts` → thread-prefix sweep; interactive-run checkpoints still never swept; startup orphan collection preserved.
13. `graphIntrospect.ts` + validator file access routed through the object store rather than a local path. (They still execute locally in this session — Session 4 moves them.)
14. Tenancy suite extended: object keys, presigned URLs, version pointers, checkpoint threads — all asserted cross-workspace-inaccessible. Add a test that a presigned URL for workspace A is rejected for a B-scoped request even if leaked.
15. Docs: storage layout, versioning model, secret lifecycle, checkpoint namespacing.

Acceptance: two server replicas behind a load balancer, no shared disk, serving generation / edit / undo / pause / resume / branch correctly with requests landing on either replica arbitrarily.

SESSION 4 — Sandboxed execution and the distributed control plane

This is the highest-risk session in the migration. Do not compress it.

The problem, stated plainly

`processManager.ts` spawns `uv run python -m jaroku_runner <agent>` as a child of the server. That subprocess executes Python written by a language model in response to a stranger's prompt. It inherits the server's network position, can read `runtime/.env`, can reach your Postgres and Redis, and can reach your cloud provider's metadata endpoint at `169.254.169.254` to steal instance credentials. The existing validator is a *quality* gate — it rejects f-string SQL and stdout writes — and it is genuinely good at that. It is not, and was never meant to be, a security boundary against a user who is deliberately trying to escape. **The sandbox is the boundary. The validator stays as a quality gate.**

Also note: `validator.ts` performs "an actual import" of the staged project, and `graphIntrospect.ts` spawns `jaroku_runner.graph`. Both execute untrusted code **on the control plane, before anything is even saved.** Both must move into the sandbox in this session. This is easy to overlook because neither is called "run".

Design

`RunSandbox` interface:

```
interface RunSandbox {
  start(spec: SandboxSpec): Promise<SandboxHandle>;
}

type SandboxSpec = {
  runId: string;
  workspaceId: string;
  image: string;                    // pinned digest, never a tag
  command: string[];
  env: Record<string, string>;      // secrets injected here, never on disk
  files: { presignedTarUrl: string };// agent version materialised at boot
  limits: { cpuMillis: number; memoryMb: number; pids: number;
            wallClockSec: number; diskMb: number };
  egress: EgressPolicy;             // computed per run, see below
  controlPlane: { url: string; runToken: string };  // short-lived, run-scoped JW
};

type SandboxHandle = {
  stdout: AsyncIterable<string>;    // NDJSON trace events
  stderr: AsyncIterable<string>;    // @@JAROKU_CTRL@@ lines + logs
  signal(sig: 'pause' | 'resume' | 'kill'): Promise<void>;
  wait(): Promise<{ exitCode: number; oom: boolean; timedOut: boolean }>;
};
```

Implementations: `LocalSubprocessSandbox` (dev only — same behaviour as today, refuses to start when `NODE_ENV=production`) and the hosted one per D4.

The egress allowlist is computed per run, and this is one of the nicer properties to fall out of the existing design. You already know exactly what an agent may talk to, because it is all declared:

- the provider endpoint for `JAROKU_PROVIDER` only (`api.anthropic.com` **or** `api.openai.com` , not both)
- the control plane URL
- the object store URL for the file fetch (or pre-materialise and drop it)
- for each selected connector: its fixed host set (`gmail.googleapis.com` , `oauth2.googleapis.com` , `slack.com`)
- for a Postgres connector: **only** the resolved host/port of the user's `DATABASE_URL` , after that URL has been validated — see below
- for each granted MCP server: exactly its host, from `mcp_tools.json`

Everything else denied. Explicitly denied even if something above resolves to them: `169.254.0.0/16` (metadata), `127.0.0.0/8` , `::1` , `10/8` , `172.16/12` , `192.168/16` , and your own VPC ranges. **Resolve DNS at policy-build time and pin the IPs**, or you have a DNS-rebinding hole: a hostname that answers with a public IP during validation and `169.254.169.254` at call time.

The `DATABASE_URL` connector is an SSRF vector and needs saying out loud. A user supplies an arbitrary Postgres connection string that your infrastructure then connects to. Validate: no private/link-local/loopback address after resolution, port allowlist, a connect timeout, and pin the resolved address into the egress policy. This is a real hole in the naive hosted version of the existing feature.

The control plane goes over the network. Today: `@@JAROKU_CTRL@@ {json}` out on stderr (this still works — the worker reads the sandbox's stderr stream), and a `<run_id>.control` *file* in. The file cannot survive a sandbox on another machine. Replace inbound with a long-poll:

```
GET  {controlPlane}/v1/runs/{runId}/control?after={seq}   Bearer <runToken>
  → 200 {"action":"pause"|"resume"|"none", "seqHigh":N}   (long-poll, ≤25s)

POST {controlPlane}/v1/runs/{runId}/mcp-confirm           Bearer <runToken>
  → blocks until allow / allow_for_run / deny / timeout(120s → DENY)
```

`runToken` is a JWT scoped to exactly one run id, valid for the run's wall-clock limit, granting nothing but these endpoints. The **timeout still denies** — preserve that exactly, including the

visible countdown in the UI and Escape-denies behaviour.

Also preserve the outside-Jaroku behaviour: `JAROKU_CONTROL_DIR` 's *absence* is how a copied-out project knows nobody is watching. In hosted mode the equivalent is the absence of `controlPlane.url` . The README's promise that the exported project runs standalone must remain literally true — add a test that exports a project and runs it with no Jaroku env at all.

Trace ingestion. Worker reads stdout line by line, parses, batches (say 50 steps or 100 ms), and writes with `INSERT … ON CONFLICT (id) DO NOTHING` , so at-least-once delivery is safe. Ordering is already solved by `seq` — keep rendering by `seq` , never arrival order, on both sides. A garbled line must not kill the run: `processManager.ts` already "survives garbled lines," and that behaviour must survive the port. Add a `trace_ingest_dropped` counter rather than silently discarding.

Backpressure. A hostile agent can emit megabytes per second on stdout. Cap: bytes-per-run, lines-per-second, and single-line length. Exceeding it is a terminal run error with a clear message, not an OOM in your worker.

15 commits

1. `RunSandbox` interface + `LocalSubprocessSandbox` refactor of `processManager.ts` , behaviour-identical, existing tests green.
2. Sandbox image: pinned Python 3.12, `uv` , pre-synced LangGraph/LangChain + connector extras, non-root user, read-only rootfs, tmpfs scratch. Reproducible build, digest-pinned.
3. `EgressPolicy` builder: from provider + connectors + MCP grants + validated `DATABASE_URL` ; DNS resolved and IP-pinned; private/link-local denied unconditionally. Heavy unit tests.
4. `DATABASE_URL` validation (resolution, private-range refusal, port allowlist, timeout) + tests with the nasty cases.
5. Hosted `RunSandbox` implementation per D4: start, stream, signal, wait, hard kill, resource limits, OOM and timeout distinguished in the result.
6. Run tokens: minting, scope, expiry, verification middleware, revocation on run completion.
7. Control-plane HTTP: the long-poll `control` endpoint; runner's `debug.py` reads it instead of the control file when a control-plane URL is present, and reads the file when it is not.

8. MCP confirmation over HTTP: blocking POST, the 120s deny-on-timeout, allow-for-run scoped to the process, denial raising as a red `tool_call` step. Every existing gate property re-tested.
9. Trace ingestion pipeline: streaming parse, batching, idempotent upsert, drop counters, garbled-line survival.
10. Backpressure caps: bytes/lines/line-length per run, terminal error path, tests with a deliberately abusive fixture agent.
11. **Move validation's import check into the sandbox.** Same 20-second kill timer, stdout captured, same rejection semantics, now with no untrusted code on the control plane. The fixture `rejected-import-time-failure.txt` must still be rejected identically.
12. **Move `graphIntrospect` into the sandbox.** Still never runs the graph. Cache the result on `agent_versions` so the graph view does not require a sandbox per view.
13. Sandbox escape test suite: attempt IMDS, attempt Postgres, attempt Redis, attempt another sandbox, attempt host FS, fork bomb, memory bomb, 10 GB stdout, DNS rebinding, `os.system` egress. Every one must fail *and be recorded*.
14. Tenancy + isolation suite extended to runs: a run's token cannot address another run; a sandbox cannot read another workspace's files or secrets.
15. Docs: the sandbox threat model, the egress policy, what the validator is and is not for.

Acceptance: the escape suite passes, a run works end to end with the sandbox on a different machine from the control plane, and pause / resume / branch / MCP-confirm all behave exactly as the README describes.

SESSION 5 — Queueing, fairness, and per-workspace limits

Goal: many workspaces sharing finite capacity without any one of them degrading the others.

Design

Queue. BullMQ on Redis, or Temporal if you want durable multi-step workflows (an eval is genuinely one). Recommend BullMQ first — the eval engine already persists its jobs to SQL and drains them, which is most of what a durable workflow buys you. **Keep `eval_jobs` as the source of truth**; the queue is a dispatch mechanism over it, not a replacement. The

existing property — "jobs are persisted before dispatch" — is exactly right and must survive.

Job classes: `run.interactive` (latency-critical), `run.eval` (throughput), `judge`, `generate`, `plan`, `edit`, `explain`, `mcp.discover`. Different concurrency, different timeouts, different retry policy.

Fairness. BullMQ has no fair queueing, and a naive global FIFO means one workspace enqueuing 500 eval jobs starves everyone. Implement a dispatcher:

- one Redis list per `(workspace_id, job_class)`
- a round-robin cursor over workspaces with pending work
- a Lua script that atomically checks the workspace's concurrency semaphore and the global one, and pops only if both admit
- workspace concurrency from `plan` (free: 1 interactive + 2 eval; paid: higher), overridable per workspace

The old caps generalise, they don't disappear. `JAROKU_EVAL_CONCURRENCY`, `JAROKU_LIMIT_<PROVIDER>` and "slot 0 reserved for the interactive run" were the single-user version of exactly this problem. Now:

- global per-provider cap (protects your platform-key rate limits — meaningless under BYOK, so make it conditional on D1)
- per-workspace per-provider cap (protects a BYOK user from their own fan-out)
- per-workspace interactive reservation (the descendant of slot 0)

Retry classification is already correct — reuse it verbatim. `test:retry` encodes transient-vs-deterministic, with unrecognised treated as deterministic. Do not let the queue's default retry policy override it; wire BullMQ's retry decision to the existing classifier.

Autoscaling. Workers scale on queue depth per class, not CPU. Publish `queue_depth`, `oldest_pending_age`, `active_sandboxes`, `sandbox_start_p95`.

Graceful shutdown. SIGTERM: stop accepting, let in-flight runs finish within a drain window, checkpoint what can be checkpointed, requeue the rest. A run killed mid-flight must land as `status: error` with a clear reason — never silence, per the existing `run_start` / `run_end` bracketing rule.

15 commits

1. Redis client, connection management, health check, `docker-compose` entry.
2. Job class definitions, payload schemas, idempotency keys.

3. Fair dispatcher: per-workspace lists, round-robin cursor, Lua admission script, unit tests for starvation and thundering-herd.
4. Concurrency semaphores: global, per-workspace, per-provider; leases with TTL so a dead worker's slot is reclaimed.
5. Worker process: separate entrypoint, drains classes it is configured for, clean shutdown.
6. `run.interactive` moved onto the queue; the per-workspace interactive reservation replaces slot 0.
7. `evalRunner.ts` fan-out onto the queue, `eval_jobs` still authoritative; existing pool tests ported.
8. Retry: existing classifier wired to the queue; bounded attempts and exponential backoff preserved; `test:retry` extended to cover queue-level retries.
9. Job timeouts and deadlines per class; `JAROKU_JOB_TIMEOUT_MS` becomes a per-class default; timeout surfaces as a traced terminal error.
10. Cancellation: `cancelEval` and a new `cancelRun` propagate to a running sandbox; in-flight jobs settle rather than vanish; interrupted evals still marked cancelled on restart.
11. Graceful shutdown + requeue + drain window; a chaos test that SIGTERMs a worker mid-eval and asserts no orphaned or duplicated jobs.
12. WebSocket fan-out over Redis pub/sub, workspace-namespaced channels; a client on replica 1 receives events for a run executing on worker 7.
13. Preserve "eval runs stay off the live trace channel" under fan-out — twenty parallel eval runs must not yank the user's timeline. Test it explicitly.
14. Load test harness: N workspaces × M concurrent runs, reporting p50/p95 dispatch latency, sandbox start time, ingestion lag, and per-workspace fairness ratio. Run it at the D6 number.
15. Docs: queue topology, fairness model, capacity planning maths from the load-test numbers.

Acceptance: at target load, one workspace submitting a 500-job eval measurably does not increase another workspace's interactive run latency.

SESSION 6 — Cost metering, budgets, billing

Goal: nobody can spend money that isn't theirs, and every number shown is one you could defend against an invoice.

Design

Reuse the existing cost model wholesale. `pricing.json` as a single source of truth read by both sides; USD per million tokens; exact-then-longest-prefix matching; cached input priced as cached input; unknown → `null`; cost summed from `steps`; partial pricing flagged. All of it is already right, and the existing test suites (`test:pricing`, `test:aggregate`) already defend it. This session adds *enforcement*, not new arithmetic.

Reservation, not post-hoc checking. With concurrency, "check the balance then spend" races: ten runs each check against the same balance and all pass. Use a hold:

```
UPDATE workspace_balances
   SET reserved = reserved + $2
 WHERE workspace_id = $1
   AND (balance - reserved) >= $2
RETURNING balance, reserved;
```

Zero rows → refuse before dispatch. Release the hold and settle actual spend at run end, in one transaction with the usage row.

Preserve the comparison/spend split. The number a provider is *compared* on counts succeeded runs only. *True spend* counts every attempt plus judge cost, and true spend is what the ceiling checks. The README is explicit about why, and getting it backwards lets a retry storm walk past the limit.

Preserve budget semantics: the ceiling bounds what is **started**, not what is spent; a job already in flight runs to completion. Do not "improve" this into a mid-run kill — you would spend the money and throw away the result.

Estimates stay honest. A range, not a point; stated basis (this agent's real runs on this model, vs. a built-in default); unpriced → `null`, never 0. The estimate informs; the ceiling enforces.

Metering table:

```
CREATE TABLE usage_events (
  id            bigserial PRIMARY KEY,
  workspace_id  uuid NOT NULL,
  run_id        text,
  kind          text NOT NULL,   -- llm.provider | llm.judge | llm.generation |
                                 -- llm.plan | llm.edit | llm.explain |
```

```
                                    -- sandbox.seconds | storage.bytes
  provider      text,
  model         text,
  input_tokens  bigint,
  output_tokens bigint,
  cached_input_tokens bigint,
  cost_usd      numeric,         -- NULL means unknown. Never 0 for unknown.
  cost_known    boolean NOT NULL,
  occurred_at   timestamptz NOT NULL DEFAULT now(),
  idempotency_key text UNIQUE
);
```

The `idempotency_key` is what makes at-least-once ingestion safe for billing.

BYOK changes what is metered, not whether. Under BYOK you still meter provider tokens (users want the dashboard) but you do not bill them — you bill sandbox seconds and storage. Keep the two concepts separate in the schema from the start.

15 commits

1. `workspace_balances`, `usage_events`, `plans`, `subscriptions` tables + repositories.
2. Plan definitions as data: concurrency, monthly credits, budget ceiling, retention, seat limits, feature flags.
3. Usage recording from trace steps, idempotent by key, cost sourced from `steps` and never `runs.cost`.
4. Usage recording for the non-run model calls: generation, plan, edit, explain, judge — each its own `kind`, judge tracked as eval overhead exactly as today.
5. Sandbox-seconds and storage-bytes metering.
6. Reservation/hold implementation with the atomic UPDATE; release-and-settle at completion; a concurrency test proving ten simultaneous runs cannot overdraw.
7. Pre-dispatch budget gate for interactive runs; refusal message names the limit and what would clear it.
8. Eval budget ceiling ported onto the reservation model; the "bounds what is started, not what is spent" semantics preserved with a test.
9. `evalEstimate.ts` extended for the hosted path; range, basis, and `null`-for-unpriced preserved.
10. BYOK path: per-workspace provider keys via `SecretStore`, injected into the sandbox environment only, validated on save with a cheap probe call, and **never** used for

platform-side generation/judging unless the workspace opts in.

11. Platform-key path with hard per-workspace ceilings and a global kill switch.
12. Stripe: checkout, webhooks (idempotent, signature-verified, replay-safe), subscription state machine, dunning, plan changes mid-cycle.
13. Client: usage dashboard, current spend vs ceiling, per-agent and per-run breakdown, cost-incomplete flagging surfaced rather than hidden.
14. Export: the existing rule holds — an export must not launder an uncertain number into a clean one. Empty cell plus `cost_known`, unscored plus the judge's reason. `test:export` and `test:csv` extended.
15. Docs: the cost model end to end, BYOK vs platform, what each plan actually limits.

Acceptance: a workspace at its ceiling cannot start a run; a workspace with ten concurrent runs cannot overdraw by a cent; every figure the dashboard shows reconciles with `usage_events`.

SESSION 7 — Connector OAuth and the credential vault

Goal: a user connects Gmail or Slack by clicking a button, and no shared secret ever exists.

Design

This is a bigger job than it looks. Today the README's requirement is that the user pastes `GMAIL_CLIENT_ID`, `GMAIL_CLIENT_SECRET`, `GMAIL_REFRESH_TOKEN` into `runtime/.env` by hand. Hosted, *you* own the OAuth app; users grant *your* app access to their account. That is a different security posture, a different consent screen, and — for Gmail — a Google verification process with a security assessment for restricted scopes. **Start the Google verification early; it has a lead time measured in weeks and it will be the long pole in this session.**

Scopes: ask for the least that makes the connector work, and match what the connector actually does.

- Gmail: the connector creates drafts and never sends. `gmail.readonly` + `gmail.compose` — **not** `gmail.send`, **not** `mail.google.com`. This matters both for the review and for the promise.

- Slack: `channels:read`, `channels:history`, `chat:write`. Posting is irreversible; the consent copy must say so, as the catalog and generation prompt already do.

Token vault. `oauth_connections` holds provider, workspace, connecting user, scopes granted, encrypted access + refresh tokens (via `SecretStore`), expiry, and status. Refresh is centralised in one module with a mutex per connection — concurrent runs refreshing the same token simultaneously is how you get a revoked refresh token with rotation enabled. Handle revocation: a 401 from a provider marks the connection `reauth_required`, notifies the workspace, and does not retry into a lockout.

Connectors get tokens the same way secrets do: injected into the sandbox environment at start, short-lived, never on disk, never in the browser. Prefer injecting a *fresh access token* with a lifetime just over the run's wall clock, so a leaked value expires fast. `gmail.py` and `slack.py` keep reading `os.environ` — their code should not need to change, which is the point of keeping the contract.

MCP at multi-tenant scale. Every MCP property in the README survives, but three things change:

- discovery becomes a queued job with its own timeouts, and must not run on the request path
- `mcp_servers` / `mcp_tools` are workspace-scoped; two workspaces may connect the same URL with different tokens and must not share a row
- an MCP server URL is user-supplied and therefore an SSRF vector: apply the same private-range refusal and IP pinning as `DATABASE_URL`, at both discovery time and call time
- the existing refusal of credential-bearing URLs, the impact ratchet, override voiding on schema change, and the one-name-one-tool refusal all stay exactly as they are

15 commits

1. `oauth_connections` + `oauth_states` tables; PKCE; state with a short TTL, single use, bound to the session.
2. Generic OAuth service: authorize URL construction, callback handling, token exchange, encrypted persistence, error classification.
3. Google OAuth: consent, incremental scopes, offline access, refresh-token rotation handling.
4. Slack OAuth v2: install flow, bot token storage, workspace mapping.
5. Centralised refresh with a per-connection mutex, proactive refresh before expiry, and

revocation → `reauth_required`.

6. Run-time credential injection: short-lived access tokens into the sandbox environment; connector Python unchanged; a test asserting no token is written to any file.
7. `catalog.json` extended with an auth mode per connector (`oauth` | `user_secret` | `none`) so the generation prompt and `.env.example` generation stay correct.
8. Postgres connector as a `user_secret` with the Session 4 URL validation, stored via `SecretStore`, egress-pinned.
9. Client: connections UI — connect, scopes shown plainly, status, disconnect, reconnect-required banner.
10. MCP registry made workspace-scoped, with per-workspace credentials; two workspaces on the same URL proven independent.
11. MCP discovery moved onto the queue; timeouts preserved; a failed refresh still never destroys a working tool list.
12. MCP URL SSRF validation and IP pinning at discovery and call time; hostile-fixture tests extended.
13. Connection revocation and workspace deletion cascade: tokens revoked at the provider, not merely deleted locally.
14. Tenancy suite extended across every connector and MCP surface.
15. Docs: OAuth setup, required app configuration, scope justifications for the Google review, and what a user is actually consenting to.

Acceptance: a new user connects Gmail and Slack with no manual `.env` editing, runs an agent that uses both, and no long-lived token ever exists outside the vault.

SESSION 8 — Hardening, abuse, data lifecycle, observability, deploy

Goal: the thing is safe to point the public at.

Design

Network posture. The README currently says, correctly: *"The server binds to localhost. It has no authentication and is not built to be exposed to a network."* That sentence is now

false and must be rewritten rather than deleted — replace it with the real posture and keep a paragraph explaining what the local-dev mode still assumes.

- CORS allowlist, credentials handling, preflight caching
- WebSocket origin enforcement (already in Session 2 — re-audit)
- CSP, HSTS, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`
- Body size limits on every endpoint; a prompt field is not unbounded
- Rate limits at three layers: edge/WAF, per-IP, per-workspace-per-action (generation and eval are expensive and need their own buckets)

Abuse. Hosted agent execution attracts crypto miners, proxy/scraping farms, and spam senders. Detect: sandbox CPU saturation with no LLM calls, egress volume anomalies, run-rate spikes, signup velocity from one IP or disposable domains, Slack-posting agents at volume. Response ladder: soft limit → require verification → suspend workspace → block. Every action audited and appealable.

Prompt-injection reality check. The README is honest that framing MCP output is *"not a defence against prompt injection — nothing is."* Keep that honesty. Multi-tenant, add: an agent's blast radius is bounded by its grants (which is already true and is the actual mitigation), high-impact confirmations still gate, and the confirmation UI shows *arguments as the body*, which is precisely the surface where an injected instruction becomes visible to a human.

Data lifecycle. Traces contain user email content, database rows, and Slack messages. That is regulated data.

- Retention per plan, enforced by a sweeper: `steps` first (the bulk), then `runs`
- `steps` partitioned by month so retention is a `DROP PARTITION` rather than a mass delete
- Export: a workspace can download everything (GDPR portability)
- Deletion: hard delete across Postgres, object store, checkpoints, and provider-side token revocation, within a stated window, with a receipt
- PII redaction option for trace payloads at ingestion for regulated customers

Observability. Structured logs with `requestId` / `workspaceId` / `runId` and a redaction filter that makes logging a secret impossible rather than merely discouraged. OpenTelemetry traces across API → queue → worker → sandbox. Metrics: run success rate, sandbox start p95, ingestion lag, queue depth by class, per-provider error rate, cost per workspace, cross-tenant-denial count (should be zero; alert on non-zero). SLOs and alerts on each.

Deploy. IaC for everything. Blue/green or rolling with health gates. Migrations run before the new version takes traffic and must be backward-compatible for one version — expand, migrate, contract, never a breaking DDL in the same deploy as the code that needs it. Backups: PITR on Postgres, versioning on the object store, and a **restore drill that you actually perform**, because an untested backup is not a backup.

15 commits

1. CORS, security headers, body limits, request timeouts.
2. Rate limiting: per-IP and per-workspace-per-action, Redis token buckets, honest `Retry-After`.
3. WAF/edge configuration as code; bot and signup-abuse rules.
4. Abuse signals: sandbox behaviour anomalies, egress volume, run-rate, signup velocity. Recorded, scored, alertable.
5. Enforcement ladder: soft limit, verification requirement, suspension, block. Audited, reversible, appealable.
6. `steps` partitioned by month; migration to move existing data; retention sweeper as a `DROP PARTITION`.
7. Retention policy per plan, applied to traces, checkpoints, staged objects, and exports.
8. Workspace data export: full archive, async job, presigned download, expiring link.
9. Workspace and account deletion: cascade across Postgres, object store, checkpoints, queue, and provider-side token revocation; receipt written to audit log.
10. Structured logging with a redaction filter and a test that asserts a known secret value cannot reach any log sink.
11. OpenTelemetry across all four tiers, with the run id as the correlating span attribute.
12. Metrics, dashboards, SLOs, alerts — including a cross-tenant-denial counter alerting on any non-zero value.
13. IaC + deploy pipeline: migrations gated ahead of traffic, expand/migrate/contract discipline documented and enforced in review.
14. Backup + PITR + object versioning + **a performed restore drill**, with the runbook written from what actually happened during it.
15. Rewrite the README for the hosted architecture: the tenancy model, the sandbox boundary, the new security notes replacing the localhost paragraph, the honest

statement of what changed about “runs on your machine” and “keys stay local,” and the local-development path that still works exactly as before.

Acceptance: an external penetration test finds no cross-tenant access and no sandbox escape; a restore drill has been completed and documented; the README describes the system that actually exists.

---

6. Things that will bite you, collected

Read this list before Session 1 and again before Session 4.

1. `validator.ts` **imports untrusted code and** `graphIntrospect.ts` **executes a Python module — both on the control plane, both before anything is saved.** Neither is called “run” and both are easy to miss.
2. **Agent slugs stop being globally unique.** Thread the uuid, not the slug.
3. `envWriter.ts` **is the only writer of** `runtime/.env` **— a single shared file.** In production nothing may write it.
4. **The MCP confirmation control file and the pause control file are local-filesystem IPC.** Both must become network calls, and the timeout must still deny.
5. **Branching physically copies a SQLite checkpoint file.** Becomes a scoped row copy; the parent-immutability guarantee must be re-proven.
6. **The relay answers reads locally with no authorisation.** That is correct for a single-user localhost app and a data breach in a hosted one.
7. `RunPool` **slot 0** is single-process fairness logic that needs re-expressing as a per-workspace reservation.
8. **User-supplied** `DATABASE_URL` **and user-supplied MCP URLs are SSRF vectors** into your own network. Validate, resolve, pin, refuse private ranges.
9. **DNS rebinding** defeats hostname-based egress rules. Pin resolved IPs.
10. **Cloud metadata at** `169.254.169.254` is the first thing a sandbox escape goes for.
11. **Client stores must fully reset on workspace switch**, or the UI leaks across tenants even when the server didn’t.
12. `SET LOCAL` **must be inside the transaction**, or RLS silently applies the wrong scope under connection pooling.

13. **At-least-once trace ingestion needs idempotent upserts**; billing needs idempotency keys. Both are cheap now and impossible to retrofit cleanly.
14. **Fixtures and the mock MCP server must keep working with no cloud dependencies**, or you lose the free development path the README is rightly proud of.
15. **Every "unknown ≠ zero" rule** — unpriced cost, unscored judge, cost-incomplete flags — has to survive three rewrites of the code around it. Keep the tests that defend them green at every commit.

---

7. What I want from you before you write code

1. Confirm D1–D6 by asking me, not by assuming.
2. Read `README.md` and `schema/events.md` in full and tell me anything in this spec that contradicts what is actually in the repository — this document was written from the README, and the code is the truth.
3. Tell me honestly whether Session 1 splits cleanly into 15 genuine commits as specified, or whether the real number is different. Do the same at the start of every session.
4. Then start Session 1, commit 1.